

氣象資料網站建議

我想要寫一個網站,資料來源是中央氣象局開放資料平台,當我開啟網頁的時候會用爬蟲抓取當天的氣象資料,然後呈現在地圖上,呈現方式想用windy那個樣子,不知道是否可行? 我需要提供什麼東西?建議秀什麼資料?

我先幫你吧可行性拆成「資料怎麼拿、地圖怎麼畫、哪些氣象圖層值得做」。這類專案重點通常不是爬蟲，而是用官方 API/開放資料穩定取得資料。

可行，而且很適合當專題或作品集。但要先修正一個觀念：**不建議用爬蟲抓中央氣象署網頁**，應該用「中央氣象署開放資料平臺 API」。官方開放資料需要註冊會員並取得 API 授權碼，使用 RESTful / GraphQL API 時要帶資料項目代碼與授權碼。 [氣象資料開放平台+1](#)

可行性判斷

你想做的可以分成三個等級：

1. 基礎版：地圖上顯示各測站氣象資料

這個最可行。

中央氣象署有「全測站逐時氣象資料」與「10 分鐘綜觀氣象資料」，資料包含測站經緯度、觀測時間、氣溫、濕度、氣壓、風速、風向、雨量、天氣現象等欄位。逐時資料更新頻率是 1 小時，10 分鐘綜觀資料更新頻率是 10 分鐘。 [氣象資料開放平台+1](#)

畫面可以像這樣：

使用者打開網頁 → 後端抓 CWA API → 整理成 GeoJSON → 前端地圖顯示測站點位 → 點擊測站看到溫度、濕度、雨量、風速、風向。

這一版最穩，也最容易完成。

2. 進階版：像 Windy 一樣有顏色熱區圖

這也可行，但會比較麻煩。

因為中央氣象署的觀測資料主要是「測站點資料」，不是每個格子都有值。所以如果你要做像 Windy 的溫度色階、雨量色階、風速色階，就需要做**空間內插**。

例如：

測站 A：28°C

測站 B：31°C

測站 C：29°C

你要把中間沒有測站的地方推估出溫度，才能畫成連續的熱力圖。

可以用：

- IDW，反距離權重，簡單好做

- Kriging，效果較好但比較難
- Delaunay / Voronoi，適合展示測站影響範圍
- 後端用 Python 處理，前端只吃結果

這版做出來就會很像「台灣即時氣象圖」。

3. 高階版：像 Windy 那種流動風場動畫

這個也能做，但要注意：只靠測站資料很難做出真正 Windy 等級的風場動畫。

Windy 那種流線動畫通常不是單純測站資料，而是來自數值天氣預報模型的格點風場資料，也就是每個網格都有 u/v 風向量。Windy 官方 Map Forecast API 本身就提供圖層、粒子動畫、圖例、等值線等能力。  Windy API

如果你只用 CWA 測站資料，也可以做「類 Windy」效果：

- 每個測站畫風向箭頭
- 依風速改變箭頭長度與顏色
- 用粒子動畫模擬風向流動
- 但準確度會比真正 Windy 低

所以我會建議你先做：

測站點位 + 風向箭頭 + 溫度/雨量熱區圖，這樣作品完成度會很高，又不會一開始就卡死在氣象模型資料處理。

你需要提供什麼？

必要

1. 中央氣象署開放資料 API 授權碼

- 到 CWA 開放資料平臺註冊會員後取得。
- 不要放在前端，要放在後端 .env。

2. 想使用的資料集代碼

- 例如：
 - 0-A0001-001：全測站逐時氣象資料
 - 0-A0003-001：10 分鐘綜觀氣象資料
- 這兩個都很適合你的需求。  氣象資料開放平台+1

3. 地圖底圖來源

- OpenStreetMap
- Leaflet
- MapLibre
- Mapbox

Leaflet 是很適合入門的開源互動地圖 JS 函式庫，輕量且支援行動裝置。  Leaflet

4. 你想做的地圖範圍

- 只做台灣本島？
- 加離島？
- 是否要顯示縣市界線？
- 是否要定位到使用者所在縣市？

5. 更新頻率

- 每次開網頁就抓一次
- 每 10 分鐘更新
- 每小時更新
- 後端快取避免一直打 API

我建議你秀哪些資料？

我會分成 MVP、進階版、展示版。

MVP 第一版

先做這些就很夠：

資料	呈現方式	價值
氣溫	測站數字 + 色階點	最直觀
雨量	藍色圓圈大小	很適合即時天氣
風速	箭頭長度 / 顏色	有 Windy 感
風向	箭頭方向	地圖視覺效果好
相對濕度	點擊測站顯示	補充資訊
氣壓	詳細面板	進階資訊
觀測時間	右上角顯示	讓資料可信

第一版畫面可以是：

左側控制面板：

- 氣溫
- 雨量
- 風速風向
- 濕度
- 測站點位

右側/地圖：

- 台灣地圖
- 測站 marker
- 不同圖層切換
- 點擊測站出現詳細資訊卡

進階版建議

1. 溫度熱力圖

把各測站溫度內插成連續色階。

例如：

- 低溫：藍色
- 舒適：綠色
- 炎熱：橘紅色

這很有作品感。

2. 雨量警示圖

用圓圈大小表示當日累積雨量：

- 0 mm：無
- 1-10 mm：小雨
- 10-40 mm：中雨
- 40 mm 以上：明顯降雨
- 80 mm 以上：警示

3. 風場箭頭圖

每個測站顯示風向箭頭：

- 箭頭方向 = WindDirection
- 箭頭大小 = WindSpeed
- 顏色 = 風速級距

這比直接做粒子動畫更實用，也比較準。

4. 測站詳細卡片

點擊測站後顯示：

測站：臺北
時間：2026-07-02 14:10

氣溫：32.1°C
 濕度：68%
 風速：3.2 m/s
 風向：東南風
 今日雨量：4.5 mm
 氣壓：1007.2 hPa

5. 右上角即時摘要

例如：

全台最高溫：臺東 35.2°C
 全台最低溫：玉山 12.4°C
 最大雨量：屏東某測站 62 mm
 最大風速：蘭嶼 12.3 m/s
 資料時間：14:10

這個很適合做 dashboard。

技術架構建議

我會推薦這個架構：

Frontend
 Next.js / React
 Leaflet or MapLibre
 地圖圖層、測站點、熱力圖、風向箭頭

Backend
 FastAPI / Next.js API Route / Node.js Express
 呼叫 CWA API
 清洗資料
 轉成 GeoJSON
 做快取

Data Source
 中央氣象署開放資料 API

Cache
 Redis / 本機 JSON 快取 / SQLite
 避免每次開網頁都直接打 CWA API

如果你想做得像專題作品，我會推薦：

Next.js + TypeScript + Tailwind + shadcn/ui + MapLibre
 後端用 Next.js API Route 或 FastAPI

如果你想快速做 MVP：

React + Leaflet + FastAPI

不建議一開始做的東西

不建議直接模仿完整 Windy

Windy 的核心難點是：

- 全球/區域格點氣象資料
- 風場向量資料
- WebGL 粒子動畫
- 時間軸動畫
- 多圖層資料管理
- 大量資料切片與快取

這些不是不能做，但對第一版來說太大。

你可以做「Windy 風格」，但不要一開始就追求 Windy 等級。

我建議你的第一版功能範圍

專案名稱可以叫：

Taiwan Weather Map | 台灣即時氣象視覺化地圖

第一版功能：

1. 開啟網頁後自動抓 CWA 觀測資料
2. 地圖顯示全台測站
3. 點擊測站看詳細資料
4. 圖層切換：
 - 氣溫
 - 雨量
 - 風速風向
 - 濕度
5. 右上角顯示更新時間
6. 左側顯示全台摘要統計
7. 手機版可用

第二版再加：

1. 溫度熱力圖
2. 雨量熱力圖
3. 風向箭頭動畫
4. 縣市篩選
5. 站點搜尋

6. 24 小時趨勢圖

第三版再加：

1. 未來天氣預報
2. 雷達回波圖層
3. 颱風路徑
4. 劇烈天氣警示
5. AI 天氣摘要

最推薦顯示的資料組合

我會建議首頁預設顯示：

風速風向 + 氣溫色階 + 測站點位

原因是這樣最像 Windy，也最有視覺效果。

圖層選單：

- 目前天氣
 - 氣溫
 - 雨量
 - 風速風向
 - 濕度
 - 氣壓
- 輔助圖層
 - 測站名稱
 - 縣市邊界
 - 地形底圖

資訊卡：

- 測站名稱
- 縣市 / 鄉鎮
- 觀測時間
- 氣溫
- 濕度
- 雨量
- 風速
- 風向
- 氣壓
- 天氣現象

結論

這個專案**非常可行**，而且蠻適合你的背景：有資料來源、有 API、有地圖視覺化、有前後端整合，也可以延伸成 AI 天氣摘要或災害預警 dashboard。

我會建議你不要說「爬蟲抓取中央氣象局資料」，而是改成：

使用中央氣象署開放資料 API，在使用者開啟網頁時由後端取得最新觀測資料，經資料清洗與格式轉換後，以互動式地圖呈現台灣即時氣象狀態，並提供類 Windy 的風速風向與氣象圖層視覺化。

這樣聽起來會專業很多，也比較符合實際開發方式。

必要
有 這個我有
想使用的資料集代碼
O-A0001-001：全測站逐時氣象資料
O-A0003-001：10 分鐘綜觀氣象資料
這兩個都很適合你的需求。
那這兩個都拿可以嗎？
地圖底圖來源
那先使用Leaflet看看
你想做的地圖範圍
台灣本島加離島
要顯示縣市界線
要定位到使用者所在縣市

更新頻率
每次開網頁或重整看後端快取 大於10分鐘抓一次這樣可以嗎
顯示更多顯示較少

可以，這樣規劃很合理，而且你現在提供的條件已經足夠做第一版了。

這兩個資料集可以都拿嗎？

可以，而且我建議兩個都拿，但用途要分清楚。

資料集	更新頻率	建議用途
O-A0003-001 10 分鐘綜觀氣象資料	10 分鐘	首頁即時地圖主要資料
O-A0001-001 全測站逐時氣象資料	1 小時	備援資料、較完整站點、趨勢/歷史感呈現

O-A0003-001 本身就包含測站名稱、經緯度、縣市鄉鎮、觀測時間、氣溫、濕度、氣壓、風速、風向、雨量、紫外線等資料，而且更新頻率是 10 分鐘，所以很適合當首頁主資料。**O-A0001-001** 是逐時資料，更新頻率 1 小時，也包含測站經緯度、氣溫、濕度、氣壓、風速風向、雨量、天氣現象等欄位，可以當備援或用來補資料。

 [氣象資料開放平台+1](#)

我的建議是：

首頁地圖優先使用 **O-A0003-001**

如果某些欄位缺值或 API 暫時失敗，再 fallback 到 0-A0001-001

不要兩份資料直接混在一起顯示，否則同一個測站可能會重複，觀測時間也可能不同。

你的更新頻率設計可以嗎？

可以，每次開網頁或重新整理時先看後端快取，如果快取超過 10 分鐘才重新抓 CWA API，這是很好的做法。

因為 0-A0003-001 更新頻率本來就是 10 分鐘，所以你設快取 10 分鐘很合理。中央氣象署開放資料 API 取用需要帶資料項目代碼與會員授權碼，做後端快取也可以避免把授權碼暴露在前端。 [氣象資料開放平台+1](#)

建議邏輯：

```

使用者打開網頁
↓
前端呼叫自己的後端 API：/api/weather/current
↓
後端檢查快取
↓
如果快取小於 10 分鐘：直接回傳快取資料
↓
如果快取大於 10 分鐘：呼叫 CWA 0-A0003-001
↓
清洗資料、轉成 GeoJSON
↓
存入快取
↓
回傳給前端 Leaflet 地圖

```

快取可以先不用 Redis，第一版用：

memory cache / local JSON file / SQLite

就夠了。

地圖範圍與功能設計

你現在的設定我會整理成這樣：

```

地圖套件：Leaflet
地圖範圍：台灣本島 + 離島
行政邊界：顯示縣市界線
定位功能：定位到使用者所在縣市
資料來源：
- 0-A0003-001：主要即時資料
- 0-A0001-001：備援或逐時資料
更新邏輯：
- 開啟網頁時讀後端快取
- 快取超過 10 分鐘才重新抓 CWA

```

Leaflet 很適合第一版，因為它輕量、好上手，也很適合做 marker、popup、圖層切換、GeoJSON 行政區邊界。

建議第一版顯示資料

我會建議首頁不要一次塞太多，先做 **4 個核心圖層**：

圖層	來源欄位	呈現方式
氣溫	AirTemperature	顏色 marker / 色階
雨量	Precipitation	圓圈大小
風速風向	WindSpeed, WindDirection	箭頭方向與長度
濕度	RelativeHumidity	popup 或切換圖層

點擊測站後再顯示完整資料：

測站名稱
縣市 / 鄉鎮
觀測時間
氣溫
相對濕度
測站氣壓
平均風速
平均風向
最大瞬間風
當日累積雨量
紫外線指數，如果有

右側或左側 dashboard 顯示全台摘要：

目前資料時間
全台最高溫測站
全台最低溫測站
最大雨量測站
最大風速測站
目前測站數量

縣市界線怎麼做？

你需要一份台灣縣市 GeoJSON。

前端 Leaflet 載入後可以這樣用：

```
county-boundary.geojson
↓
Leaflet L.geoJSON()
↓
畫出縣市邊界
↓
點擊縣市可以 zoom in
```

這份資料可以放在：

`/public/data/taiwan-counties.geojson`

第一版不需要每次從網路抓縣市界線，直接放靜態檔即可。

使用者定位到所在縣市怎麼做？

可以用瀏覽器 Geolocation API：

```
navigator.geolocation.getCurrentPosition()
```

取得使用者經緯度後，有兩種做法：

簡單版

直接把地圖移動到使用者座標附近。

```
map.setView([lat, lng], 10)
```

進階版

用使用者座標判斷落在哪一個縣市 polygon 裡，然後 highlight 該縣市。

```
使用者經緯度
↓
turf.js point-in-polygon
↓
找出所在縣市
↓
地圖 zoom 到該縣市
↓
高亮縣市邊界
```

建議第一版先做簡單版，第二版再做縣市判斷。

資料處理建議

你後端不要直接把 CWA 原始 JSON 丟給前端，建議整理成自己的格式。

例如轉成 GeoJSON：

```
JSON
{
  "type": "FeatureCollection",
  "updatedAt": "2026-07-02T14:10:00+08:00",
  "source": "CWA 0-A0003-001",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [121.5148, 25.0377]
      },
      "properties": {
        "stationId": "466920",
        "stationName": "臺北",

```

```

    "county": "臺北市",
    "town": "中正區",
    "observedAt": "2026-07-02T14:10:00+08:00",
    "temperature": 32.1,
    "humidity": 68,
    "pressure": 1007.2,
    "windSpeed": 3.2,
    "windDirection": 120,
    "precipitation": 0.5,
    "uvi": 6
  }
}
]
}

```

這樣前端會很好畫。

建議專案架構

如果你用 Next.js，我會這樣做：

```

weather-map/
├── app/
│   ├── page.tsx
│   └── api/
│       └── weather/
│           ├── current/
│           └── route.ts
├── components/
│   ├── WeatherMap.tsx
│   ├── WeatherLayerControl.tsx
│   ├── WeatherStationPopup.tsx
│   └── WeatherSummaryPanel.tsx
├── lib/
│   ├── cwa.ts
│   ├── weather-cache.ts
│   ├── weather-transform.ts
│   └── color-scale.ts
├── public/
│   └── data/
│       └── taiwan-counties.geojson
├── .env.local
└── package.json

```

.env.local：

```

env
CWA_API_KEY=你的中央氣象署授權碼
CWA_PRIMARY_DATASET=0-A0003-001
CWA_FALLBACK_DATASET=0-A0001-001
WEATHER_CACHE_TTL_SECONDS=600

```

我會建議你的第一版開發順序

第一階段：資料打通

先不要畫漂亮地圖，先讓後端成功回傳整理後的 GeoJSON。

```
/api/weather/current
```

要確認：

```
有幾個測站
資料時間
每個測站有沒有經緯度
氣溫/雨量/風速是否有成功轉成數字
```

第二階段：Leaflet 顯示測站

地圖上顯示：

```
marker
popup
不同氣溫顏色
```

第三階段：加入圖層切換

```
氣溫
雨量
風速風向
濕度
```

第四階段：縣市界線

加入：

```
台灣本島 + 離島縣市 GeoJSON
縣市 hover
縣市點擊 zoom
```

第五階段：類 Windy 視覺

加入：

```
風向箭頭
色階圖例
全台摘要面板
右上更新時間
```

最終建議版本

你現在的規格可以定成：

```
使用中央氣象署開放資料 0-A0003-001 作為主要即時資料來源，0-A0001-001 作為備援資料來源。使用者開啟網站時，前端呼叫後端 API，後端先檢查快取，若資料超過 10 分鐘才重新向 CWA API 取得最新觀測資料。資料經清洗後轉成 GeoJSON，並透過 Leaflet 顯示在台灣本島與離島地圖上。地圖包含縣市界線、使用者定位、測站資訊 popup、氣溫色階、雨量大小、風速風向箭頭與圖層切換，呈現類 Windy 的即時氣象視覺化效果。
```

這樣是非常合理的第一版規格。

下一步可以直接把這段整理成給 Claude / Antigravity agent 的開發 prompt。

可以給我一個給claude的 prompt嗎

可以，下面這份可以直接貼給 Claude / Antigravity agent 使用。我會寫成「已有專案也能用、沒有專案也能建立」的版本，並明確要求先規劃再開發。

這份可以直接丟給 Claude。

比較建議你第一次先讓它只做 **MVP**，不要一次叫它把 Windy 動畫、縣市高亮、所有圖層都做完，會比較穩。

抓資料部分可以使用爬蟲

可以，但我會建議你在 prompt 裡面寫成：

「優先使用官方 API，如果 API 欄位不足或資料格式不方便，允許加入爬蟲作為備援或補充，但不要把爬蟲當第一資料來源。」

原因是：

1. 官方 API 比爬蟲穩定

- 格式固定
- 不容易因網頁改版壞掉
- 比較符合專題/作品集的工程實務

2. 爬蟲可以當補充

- 有些頁面資料 API 不一定好找
- 有些圖層或圖片型資料，例如雷達回波、颱風圖、警特報頁面，可能爬網頁會比較直覺
- 但要注意不要高頻率請求，也不要爬需要登入或禁止爬取的內容

3. 你的專案主軸還是地圖視覺化

- 如果把重點放在爬蟲，專案容易變成「抓資料專案」
- 如果用 API + 快取 + 地圖渲染，會比較像正式產品

你可以把原本 prompt 裡這段：

改成：

然後原本注意事項裡這一點：

改成：

我的建議是：**觀測站資料用 API，圖像型資料或不好拿的補充資料才用爬蟲**。這樣最穩，也最合理。